GUIDE ADMINISTRATEUR

Kardo — Administration & Déploiement

Hébergement, configuration, gestion des commerçants, maintenance.

Public : Administrateur Kardo **Version :** 1.0 **Mise à jour :** Avril 2026

Sommaire

1. Hébergement — Railway, Hetzner, Render
2. Variables d'environnement
3. Déploiement production (checklist)
4. Configuration des certificats Apple & Google
5. Configuration Supervisor (queue + scheduler)
6. Gestion des commerçants (pros)
7. Limites par plan
8. Domaine, DNS, Resend
9. Maintenance courante
10. Commandes artisan utiles
11. Audit de sécurité (état + actions)
12. Dépannage

1. Hébergement — Railway, Hetzner, Render

Option 1 — Railway (recommandé pour démarrer)

Prix : 0 $ → ~5 $ / mois au début, scale à la demande.

- PostgreSQL inclus (gratuit jusqu'à 1 GB)
- Deploy automatique depuis GitHub
- SSL automatique
- Queue worker intégré dans le même container

```
# 1. Créer un compte sur railway.app
# 2. Connecter le repo GitHub
# 3. Ajouter un service PostgreSQL
# 4. Configurer les variables d'environnement (voir section 2)
# 5. Push sur main → deploy auto

# Railway CLI (optionnel)
npm install -g @railway/cli
railway login && railway init && railway up
```

Option 2 — Hetzner VPS (le moins cher long terme)

Prix : 4.51 € / mois (CX22 — 2 vCPU, 4 GB RAM, 40 GB SSD).

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y php8.4 php8.4-fpm php8.4-pgsql php8.4-mbstring \
  php8.4-xml php8.4-curl php8.4-gd php8.4-zip nginx postgresql \
  nodejs npm supervisor certbot

sudo -u postgres createuser kardo -P
sudo -u postgres createdb kardo -O kardo

# Cloner le repo, configurer .env, php artisan migrate, npm run build
# Configurer supervisor pour queue:work et schedule:work
# Certbot pour SSL
```

Option 3 — Render

Prix : 0 $ (free tier) → 7 $ / mois (starter). Similaire à Railway, avec un free tier généreux.

Quand scaler ?

Nb pros	Nb clients total	Hébergement recommandé

1-10	< 5 000	Railway free / Hetzner CX22
10-50	5 000-50 000	Hetzner CX32 (8 € / mois)
50-200	50 000-200 000	Hetzner CX42 + PostgreSQL dédié
200+	200 000+	CX52 + PostgreSQL dédié + Redis cache

2. Variables d'environnement

Toutes les variables sont à définir dans le panel Railway (ou `.env` en local / Hetzner).

```
APP_NAME=Kardo
APP_ENV=production
APP_KEY=base64:...                # généré avec: php artisan key:generate
APP_DEBUG=false
APP_URL=https://kardo.fr
APP_TIMEZONE=Europe/Paris
APP_LOCALE=fr

# Base de données
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1                 # ou URL du service PostgreSQL Railway
DB_PORT=5432
DB_DATABASE=kardo
DB_USERNAME=kardo
DB_PASSWORD=...                   # mot de passe fort

# Queue
QUEUE_CONNECTION=database

# Apple Wallet
APPLE_PASS_TYPE_ID=pass.fr.kardo
APPLE_TEAM_ID=6DKW9Y76F7
APPLE_CERT_PATH=storage/certs/pass-cert.p12
APPLE_CERT_PASSWORD=...
PASS_CERT_P12_B64=...             # certificat .p12 encodé en base64
WWDR_CERT_B64=...                 # WWDR cert encodé en base64
```

```
# Google Wallet
GOOGLE_WALLET_ISSUER_ID=3388000000023095407
GOOGLE_SERVICE_ACCOUNT_JSON=storage/certs/google-service-account.json
GOOGLE_SERVICE_ACCOUNT_B64=...     # JSON service account encodé en base64

# Firebase (push Android)
FCM_PROJECT_ID=kardo-492108

# Email (Resend)
MAIL_MAILER=resend
MAIL_FROM_ADDRESS=jaggyinkgraph@gmail.com
MAIL_FROM_NAME=Kardo
RESEND_API_KEY=re_...
```

Attention : les variables `*_B64` sont obligatoires en prod sur Railway car les fichiers .p12 et .json ne peuvent pas être committés dans le repo. Le script `docker-start.sh` les décode au démarrage.

3. Déploiement production (checklist)

```
# 1. Cloner le repo
git clone <repo> kardo-app && cd kardo-app

# 2. Installer les dépendances
composer install --optimize-autoloader --no-dev
npm install && npm run build
cd node-services && npm ci --omit=dev && cd ..

# 3. Configurer l'environnement
cp .env.example .env
# Éditer .env avec les valeurs de production

# 4. Générer la clé d'app
php artisan key:generate

# 5. Migrer la base
php artisan migrate --force
```

```
# 6. Cacher la config
php artisan config:cache
php artisan route:cache
php artisan view:cache

# 7. Créer le symlink storage
php artisan storage:link

# 8. Copier les certificats dans storage/certs/
# pass-cert.p12, pass-signer-cert.pem, pass-signer-key.pem
# google-service-account.json, wwdr-cert.pem

# 9. Lancer les workers (via Supervisor en prod — voir section 5)
```

4. Configuration des certificats Apple & Google

Apple Pass Type ID + certificat .p12

1. Aller sur developer.apple.com → Certificates, Identifiers & Profiles
2. Identifiers → + → Pass Type IDs → créer `pass.fr.kardo`
3. Certificates → + → Pass Type ID Certificate → choisir `pass.fr.kardo`
4. Générer un CSR avec Keychain (Mac) ou OpenSSL (Linux)
5. Télécharger le certificat `.cer`, l'importer dans Keychain
6. Exporter en `.p12` avec un mot de passe fort
7. Encoder en base64 : `base64 -w0 pass-cert.p12 > pass-cert.p12.b64`
8. Coller le contenu dans la variable d'env `PASS_CERT_P12_B64` sur Railway

Apple WWDR Intermediate Certificate

1. Télécharger depuis apple.com/certificateauthority → "Worldwide Developer Relations - G4"
2. Le fichier est en format DER, encoder en base64 : `base64 -w0 AppleWWDRCAG4.cer > wwdr.b64`
3. Coller dans `WWDR_CERT_B64`

Google Wallet Service Account

1. Aller sur console.cloud.google.com → projet `kardo-492108`
2. Activer l'API "Google Wallet API"
3. IAM & Admin → Service Accounts → créer `kardo-wallet@kardo-492108.iam.gserviceaccount.com`
4. Rôles : "Google Wallet API Service Agent"
5. Clés → Ajouter une clé → JSON → télécharger
6. Encoder en base64 : `base64 -w0 google-service-account.json > sa.b64`
7. Coller dans `GOOGLE_SERVICE_ACCOUNT_B64`
8. Aller sur pay.google.com/business/console pour récupérer l'**Issuer ID**

Firebase FCM (push Android)

1. Lié au même projet Google Cloud `kardo-492108`
2. Le FCM v1 utilise OAuth2 — le service account ci-dessus suffit (rôle "Firebase Cloud Messaging API Admin" supplémentaire)
3. Définir `FCM_PROJECT_ID=kardo-492108`

5. Configuration Supervisor (queue + scheduler)

Sur Railway, le script `docker-start.sh` lance directement queue:work + schedule:run en background. Sur un VPS (Hetzner), il faut configurer Supervisor.

```
# /etc/supervisor/conf.d/kardo-worker.conf

[program:kardo-queue]
process_name=%(program_name)s_%(process_num)02d
command=php /var/www/kardo/artisan queue:work --sleep=3 --tries=3 --max-time=3600
autostart=true
autorestart=true
stopasgroup=true
killasgroup=true
numprocs=2
user=www-data
redirect_stderr=true
stdout_logfile=/var/www/kardo/storage/logs/worker.log
```

```
[program:kardo-scheduler]
command=/bin/bash -c "while true; do php /var/www/kardo/artisan schedule:run --no-i
autostart=true
autorestart=true
user=www-data
redirect_stderr=true
stdout_logfile=/var/www/kardo/storage/logs/scheduler.log
```

```
sudo supervisorctl reread
sudo supervisorctl update
sudo supervisorctl start kardo-queue:*
sudo supervisorctl start kardo-scheduler
```

6. Gestion des commerçants (pros)

Lister les pros

```
php artisan kardo:merchants
# Affiche : nom, email, plan, nombre de cartes
```

Créer un pro manuellement

```
php artisan tinker
```

```
App\Models\Merchant::create([
    'name' => 'Boulangerie Martin',
    'email' => 'contact@boulangerie-martin.fr',
    'password' => bcrypt('motdepasse-temporaire'),
    'plan' => 'solo',  // free, solo, pro, business
    'pass_design' => [
        'program_name' => 'Carte Boulangerie Martin',
        'reward_label' => '1 baguette offerte',
        'max_stamps' => 10,
        'background_color' => '#D4A574',
        'foreground_color' => '#FFFFFF',
```

```
        'label_color' => '#F5E6D3',
    ],
]);
```

Changer le plan d'un pro

```
php artisan kardo:set-plan contact@boulangerie-martin.fr pro
```

Connexion psql Railway

```
psql "postgresql://postgres:xxx@ballast.proxy.rlwy.net:38503/railway"

UPDATE merchants SET plan = 'pro' WHERE email = 'email@commerce.fr';
```

Supprimer un pro et toutes ses données

```
php artisan tinker
```

```
$merchant = App\Models\Merchant::where('email', 'email@example.com')->first();

$merchant->cards()->each(function ($card) {
    $card->purchases()->delete();
    $card->rewards()->delete();
    $card->pushTokens()->delete();
    $card->forceDelete();
});
$merchant->notifications()->delete();
$merchant->enrollmentLinks()->delete();
$merchant->rewardTiers()->delete();
DB::table('audit_logs')->where('merchant_id', $merchant->id)->delete();
DB::table('automation_rules')->where('merchant_id', $merchant->id)->delete();
$merchant->delete();
```

Attention : cette opération est **irréversible**. Toutes les cartes Wallet du commerçant seront invalidées et les clients ne pourront plus les utiliser.

7. Limites par plan

Feature	Free	Solo (19 €)	Pro (39 €)	Business (79 €)
Clients max	50	300	1 500	Illimité
Broadcasts / mois	0	2	Illimité	Illimité
Automations	0	1	4	Illimité
CRM avancé	Non	Non	Oui	Oui
Export CSV	Non	Non	Oui	Oui
Journal d'audit	Non	Non	Oui	Oui

Les limites sont définies dans `config/kardo.php` et appliquées :

- **Côté serveur :** middleware de plan + validation dans les controllers
- **Côté client :** composable `usePlan()` qui masque les features non accessibles

8. Domaine, DNS, Resend

Acheter le domaine

- `kardo.fr` sur OVH (~7 € / an) ou Namecheap

DNS

```
# Si Hetzner / VPS classique :
A     @       → IP du serveur
A     www     → IP du serveur

# Si Railway :
CNAME @       → kardo-production.up.railway.app
CNAME www     → kardo-production.up.railway.app
```

Resend — vérifier le domaine

1. resend.com/domains → Add domain → `kardo.fr`
2. Ajouter les records DNS fournis par Resend (SPF, DKIM, DMARC)
3. Attendre la vérification (~10 min)

> **Important :** tant que Resend n'est pas vérifié, les emails partent avec un domaine de fallback et risquent d'atterrir en spam.

9. Maintenance courante

Sauvegardes base de données

```
# Backup quotidien (cron)
pg_dump -U kardo kardo > /backups/kardo-$(date +%Y%m%d).sql

# Restaurer
psql -U kardo kardo < /backups/kardo-20260414.sql
```

Mise à jour du code (Hetzner)

```
cd /var/www/kardo
git pull origin main
composer install --optimize-autoloader --no-dev
npm install && npm run build
php artisan migrate --force
php artisan config:cache && php artisan route:cache && php artisan view:cache
sudo supervisorctl restart kardo-queue:*
```

Sur Railway : push sur `main` = redéploiement auto (~3 min).

Logs

```
# Logs Laravel
tail -f storage/logs/laravel.log
```

```
# Logs queue worker (Hetzner)
tail -f storage/logs/worker.log

# Logs Railway
railway logs
```

RGPD : nettoyage automatique

La commande `kardo:cleanup-inactive` tourne tous les jours à 03h00 et anonymise les cartes inactives depuis > 3 ans. Rien à faire manuellement.

10. Commandes artisan utiles

Commande	Description	Planification
`kardo:merchants`	Liste tous les commerçants	Manuel
`kardo:set-plan {email} {plan}`	Change le plan d'un commerçant	Manuel
`kardo:process-automations`	Exécute les règles d'autopilot	Toutes les heures
`kardo:process-scheduled`	Envoie les broadcasts programmés	Toutes les minutes
`kardo:cleanup-inactive`	Anonymise les cartes inactives > 3 ans (CNIL)	Quotidien à 03h00

```
# Debug
php artisan queue:work --tries=1              # Lancer le queue worker manuellemer
php artisan schedule:work                     # Lancer le scheduler manuellement
php artisan tinker                            # Console interactive

# Build
npm run build                                 # Build les assets Vue
```

```
php artisan config:cache                        # Cacher la config
php artisan route:cache                         # Cacher les routes
```

11. Audit de sécurité (état + actions)

Protégé

Élément	Status	Détail
Authentification	OK	Sanctum bearer tokens, 24h expiration
Mots de passe	OK	bcrypt 12 rounds, validation maj+min+chiffre
Isolation tenant	OK	Ownership check sur chaque route
Rate limiting	OK	5 / min login, 60 / min API
SQL injection	OK	Eloquent, pas de raw queries
XSS	OK	Vue.js échappe par défaut, pas de v-html
Upload fichiers	OK	Validation MIME, SVG bloqué, max 4 MB
CSV injection	OK	Préfixage des cellules à risque
HTTPS	OK	URL::forceScheme + Railway SSL
Secrets dans repo	OK	.gitignore + env vars base64
RGPD	OK	Consentement, droits, anonymisation auto

À améliorer

Élément	Risque	Action
APP_KEY de prod	Moyen	Régénérer une APP_KEY unique pour la prod
Pas de CSP headers	Faible	Ajouter Content-Security-Policy

Pas de 2FA pro	Faible	Ajouter authentification 2 facteurs
LOG_LEVEL en prod	Faible	Passer de debug à warning
Cert Apple expiration	Info	14/05/2027 — prévoir renouvellement

12. Dépannage

Les passes Apple ne se mettent pas à jour

- Vérifier que le queue worker tourne : `railway logs | grep queue`
- Vérifier le certificat .p12 : valide ? bon mot de passe ?
- Vérifier la timezone : `config('app.timezone')` = `Europe/Paris` ?
- Tester l'endpoint Apple : `curl -I https://kardo-production.up.railway.app/api/v1/...`

Les notifications Android n'arrivent pas

- Vérifier `FCM_PROJECT_ID` et le service account JSON
- Vérifier que le client a bien autorisé les notifs Google Wallet
- Logs : `tail -f storage/logs/laravel.log | grep FCM`

Les inscriptions ne s'enregistrent pas

- Vérifier le slug dans la table `enrollment_links`
- Vérifier la limite du plan : `php artisan kardo:merchants`
- Logs PostgreSQL pour repérer une violation de contrainte unique

Le serveur ne démarre pas

- Vérifier que `docker-start.sh` a bien décodé les certs (logs au démarrage)
- Vérifier `APP_KEY` et `APP_DEBUG=false`
- Tester localement avec `composer run dev`